

Chapter 8: Stochastic Methods

Algorithms for Optimization, 2nd ed. 2026

Kochenderfer & Wheeler

Lecture Notes

2026

Chapter Outline

- 1 Introduction to Stochastic Methods
- 2 8.1 Noisy Descent
- 3 8.2 Mesh Adaptive Direct Search (MADS)
- 4 8.3 Memory-Efficient Zeroth-Order Optimization (MeZO)
- 5 8.4 Simulated Annealing
- 6 8.5 Cross-Entropy Method
- 7 8.6 Proposal Distribution Descent
- 8 8.7 Information-Geometric Optimization (IGO)
- 9 8.8 Covariance Matrix Adaptation (CMA-ES)
- 10 Method Comparison
- 11 8.9–8.10 Summary & Exercises
- 12 Conclusion

Why Stochastic Methods?

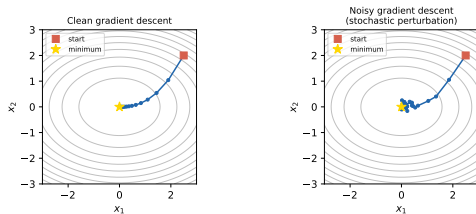
Motivation

- **Escape local minima:** deterministic gradient methods can get stuck.
- **Noisy evaluations:** objective may not be computed exactly (neural network training, simulation-based optimisation).
- **Randomisation increases coverage** of the design space.
- **Large-scale models:** zeroth-order methods reduce memory cost when gradients are expensive or unavailable.

Stochastic vs. Deterministic

Property	Deterministic	Stochastic
Local minima	stuck	can escape

Effect of Noise on Gradient Descent



Clean vs. noisy gradient descent on a quadratic bowl. Noise can help escape narrow valleys.

8.1 Noisy Descent

Stochastic Gradient Descent (SGD)

Adding stochasticity to gradient descent:

$$\mathbf{x}^{(k+1)} \leftarrow \mathbf{x}^{(k)} - \alpha^{(k)} \tilde{\nabla} f(\mathbf{x}^{(k)})$$

where $\tilde{\nabla} f$ is the *noisy gradient* with

$$\tilde{\nabla} f(\mathbf{x}) = \nabla f(\mathbf{x}) + \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})$$

Convergence Conditions (Robbins–Monro)

$$\sum_{k=1}^{\infty} \alpha^{(k)} = \infty, \quad \sum_{k=1}^{\infty} \left(\alpha^{(k)}\right)^2 < \infty$$

- The step must be large enough to reach the minimum.

Why Noise Helps

- Flat regions: noise provides a random kick.
- Narrow valleys: escapes with probability $\propto e^{-\Delta/\sigma^2}$.
- Neural-network training: mini-batches provide cheap noisy gradients.

Caveat

The gradient estimator must be **unbiased**:

$$\mathbb{E}[\tilde{\nabla} f(\mathbf{x})] = \nabla f(\mathbf{x})$$

Otherwise the method converges to the wrong point.

8.1 Noisy Descent – Python Implementation

```
1 import numpy as np
2
3 def noisy_descent(f, grad_f, x0, alpha=lambda k: 1.0/k,
4                  sigma=0.1, max_iter=1000, tol=1e-5):
5     """
6     f          : objective function    R^n -> R
7     grad_f     : gradient function    R^n -> R^n
8     x0         : initial point
9     alpha      : step-size schedule   k -> float
10    sigma      : noise standard deviation
11    """
12    x = np.array(x0, dtype=float)
13    for k in range(1, max_iter + 1):
14        g = grad_f(x)
15        noise = np.random.randn(len(x)) * sigma
16        g_hat = g + noise # noisy gradient
17        x = x - alpha(k) * g_hat
18        if np.linalg.norm(alpha(k) * g_hat) < tol:
19            break
20    return x
21
22 # Example: minimize f(x) = x1^2 + 2*x2^2 from x0 = [3, 3]
23 f = lambda x: x[0]**2 + 2*x[1]**2
24 grad_f = lambda x: np.array([2*x[0], 4*x[1]])
25 x_opt = noisy_descent(f, grad_f, [3.0, 3.0], sigma=0.5)
26 print(f"Optimum: {x_opt}") # approx [0, 0]
```

8.2 Mesh Adaptive Direct Search (MADS)

7.4. GENERALIZED PATTERN

Key Idea

- **Pattern search** method that uses random directions.
- Generates a *positive spanning set* of directions at each iteration.
- Probes $n + 1$ directions from a random lower-triangular matrix $\mathbf{L}$.
- Step size α starts at 1, always a power of 4, $\alpha \leq 1$.

dimensions. The Hooke-Jeeves method is susceptible to local minima. The method has been proven to converge on certain classes of functions.⁴

```
function hooke_jeeves(f, x, alpha, epsilon, gamma=0.5)
    y, n = f(x), length(x)
    while alpha > epsilon
        improved = false
        best = (x=x, y=y)
        for i in 1:n
            for sgn in (-1,1)
                x' = x + sgn*alpha*basis(i, n)
                y' = f(x')
                if y' < best.y
                    best, improved = (x=x', y=y'), true
                end
            end
        end
        x, y = best
        if !improved
            alpha *= gamma
        end
    end
    return x
end
```

⁴E. D. Dolan, R. Torczon, "On the convergence of Pattern Search", *Journal on Optimization* no. 2, pp. 567-58

Algorithm 7.5. The method, which function f , a starting step size α , a step decay γ . Until the step size the points sample directions improvement. Based on Algorithm 178: *Communications of the* pp. 313-314, 1961

Positive Spanning Set Construction

- 1 $\delta = \lfloor 1/\sqrt{\alpha} \rfloor$
- 2 Build random lower-triangular $\mathbf{L}$ with diagonal ± 1 and off-diagonal entries in $[-\delta + 1, \delta - 1]$
- 3 Randomly permute rows and columns $\Rightarrow \mathbf{D}$

Example 8.1. Eight positive spanning sets for $\mathbb{R}^2$. Only the lower-triangular generation can produce the first two columns.

8.2 MADDS – Step-Size Update Rule

Algorithm 8.3: MADDS Update

- 1 Evaluate $f(\mathbf{x}^{(k)} + \alpha \mathbf{d})$ for each direction $\mathbf{d}$ in the spanning set.
- 2 If improvement found at $\mathbf{x}' = \mathbf{x} + \alpha \mathbf{d}$:
 - Mark improved = True
 - Also probe $\mathbf{x}'' = \mathbf{x} + 3\alpha \mathbf{d}$ (opportunistic step)
 - Accept the better of $\mathbf{x}', \mathbf{x}''$
- 3 Update step size:

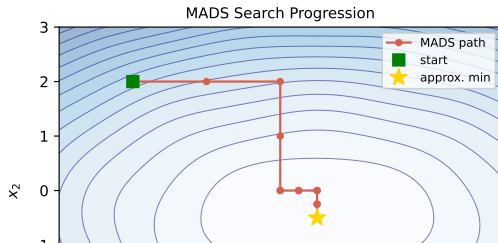
$$\alpha^{(k+1)} \leftarrow \begin{cases} \min(1, 4\alpha^{(k)}) & \text{if improved} \\ \alpha^{(k)}/4 & \text{otherwise} \end{cases}$$

Convergence Property

Using a power of 4 means the maximum componentwise step size $\alpha/\sqrt{\alpha} = \sqrt{\alpha}$ stays on a grid of spacing 2^m for integer $m \leq 1$.

Properties

- **Convergence guaranteed** when $\alpha \rightarrow 0$ (dense patterns).
- **No gradient required** – purely zeroth-order.
- **Opportunistic**: not dynamic-ordering (direction after success is random).
- Practical for **noisy, black-box** objectives.



8.2 MADS – Python Implementation

```
1 import numpy as np
2
3 def rand_positive_spanning_set(alpha, n):
4     """Algorithm 8.2: random positive spanning set of n+1 directions."""
5     delta = int(round(1 / np.sqrt(alpha)))
6     L = np.diag(np.random.choice([-1, 1], n))      # diagonal +/-1
7     for i in range(1, n):
8         for j in range(i):
9             L[i, j] = np.random.randint(-delta+1, delta) # subdiagonal
10    perm_r = np.random.permutation(n)
11    perm_c = np.random.permutation(n)
12    D = L[perm_r, :][:, perm_c]
13    extra = -D.sum(axis=1, keepdims=True)           # -(sum of columns)
14    return np.hstack([D, extra]).T                  # shape (n+1, n)
15
16 def mesh_adaptive_direct_search(f, x, eps=1e-4, alpha_init=1.0):
17     """Algorithm 8.3: MADS."""
18     alpha, y, n = alpha_init, f(x), len(x)
19     while alpha > eps:
20         improved = False
21         for d in rand_positive_spanning_set(alpha, n):
22             x_new = x + alpha * d; y_new = f(x_new)
23             if y_new < y:
24                 x, y, improved = x_new, y_new, True
25                 x2 = x + 3*alpha*d; y2 = f(x2)      # opportunistic probe
26                 if y2 < y: x, y = x2, y2
27                 break
```


8.3 Memory-Efficient Zeroth-Order Optimization

Motivation

- Optimising LLMs with **billions of parameters**.
- Backpropagation requires storing all activations: $O(n)$ memory for gradients alone.
- **MeZO** (Zhang et al., 2022): estimate gradient using *two* forward passes with a PRNG seed.

MeZO Gradient Estimator

Sample $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ using a **fixed seed** s :

$$\hat{\mathbf{g}} = \frac{f(\mathbf{x} + \epsilon \mathbf{z}) - f(\mathbf{x} - \epsilon \mathbf{z})}{2\epsilon} \mathbf{z}$$

Update: $\mathbf{x} \leftarrow \mathbf{x} - \alpha \hat{\mathbf{g}}$

Re-generate $\mathbf{z}$ using the same seed s to apply the update *in-place* without storing $\mathbf{z}$ explicitly.

Key Properties

- **Memory**: $O(1)$ extra (two scalar function evaluations).
- **Unbiased**: $\mathbb{E}[\hat{\mathbf{g}}] = \nabla f(\mathbf{x})$ (spherical symmetry).
- **No backward pass** required.
- Suitable for **fine-tuning** large frozen models.

Numerical Example (1D)

$f(x) = x^2$, $x_0 = 3$, $\epsilon = 0.3$, $\alpha = 0.1$:

$$\hat{g} = \frac{(3 + 0.3z)^2 - (3 - 0.3z)^2}{0.6} z = \frac{3.6 \cdot 0.3z^2}{0.6}$$

For $z = 1$: $\hat{g} = 6.0$ (true grad = 6). ✓

8.3 MeZO – Python Implementation

```
1 import numpy as np
2
3 def mezo_step(f, x, eps=1e-3, alpha=1e-3, seed=None):
4     """
5     One MeZO step. Re-uses PRNG seed to avoid storing z.
6     f      : objective (x -> scalar)
7     x      : current parameters (modified in place)
8     eps    : smoothing radius
9     alpha  : learning rate
10    """
11    rng = np.random.default_rng(seed)
12    z = rng.standard_normal(x.shape)          # random direction
13    # Two function evaluations (forward passes only)
14    f_plus = f(x + eps * z)
15    f_minus = f(x - eps * z)
16    grad_est = (f_plus - f_minus) / (2 * eps)  # scalar
17    # Re-generate same z using same seed for in-place update
18    rng2 = np.random.default_rng(seed)
19    z2 = rng2.standard_normal(x.shape)
20    x -= alpha * grad_est * z2                # O(1) extra memory
21    return x
22
23 def mezo_optimize(f, x0, n_iter=500, eps=1e-3, alpha=1e-3):
24     x = np.array(x0, dtype=float)
25     for k in range(n_iter):
26         x = mezo_step(f, x, eps=eps, alpha=alpha, seed=k)
27     return x
```

8.4 Simulated Annealing – Core Idea

Physical Analogy

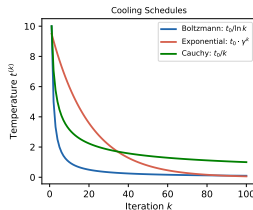
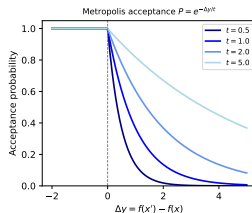
Inspired by the cooling of metals in metallurgy:

- High temperature $\Rightarrow$ high randomness, explores widely.
- Low temperature $\Rightarrow$ low randomness, exploits locally.
- **Annealing schedule:** gradually reduce temperature $t^{(k)}$.

Metropolis Acceptance Criterion

At iteration k , propose $\mathbf{x}' \sim T(\mathbf{x}'|\mathbf{x}^{(k)})$:

$$\mathbf{x}^{(k+1)} \leftarrow \begin{cases} \mathbf{x}' & \text{if } f(\mathbf{x}') \leq f(\mathbf{x}^{(k)}) \\ \mathbf{x}' & \text{with prob. } e^{-\Delta y / t^{(k)}} \\ \mathbf{x}^{(k)} & \text{otherwise} \end{cases}$$



Left: acceptance probability vs. Δy for various temperatures. Right: three standard cooling schedules.

8.4 Simulated Annealing – Cooling Schedules

Standard Cooling Schedules

Name	Schedule	Convergence
Boltzmann	$t^{(k)} = t^{(1)} / \ln(k)$	Slow, guaranteed
Cauchy	$t^{(k)} = t^{(1)} / k$	Fast, not guaranteed
Exponential	$t^{(k)} = t^{(1)} \gamma^k$	Practical

Boltzmann Annealing

- Uses Gaussian transition: $\mathbf{x}' = \mathbf{x} + t^{(k)} \mathcal{N}(\mathbf{0}, \mathbf{I})$
- Converges to global minimum *asymptotically*.
- Requires $t^{(1)} / \ln(k)$ cooling $\Rightarrow$ very slow in practice.

Fast Simulated Annealing (Cauchy)

- Cauchy/Lorentz transition distribution.

Numerical Example

$t^{(1)} = 10$, Exponential $\gamma = 0.95$:

k	$t^{(k)}$
1	10.00
10	5.99
50	0.769
100	0.059
200	0.00035

Acceptance probability at $t = 1$

$$P(\Delta y = 1) = e^{-1/1} \approx 0.368$$

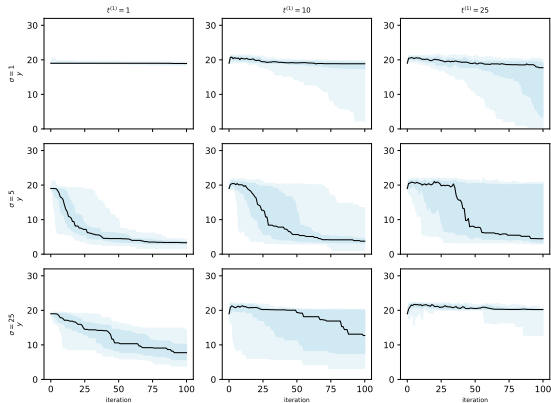
$$P(\Delta y = 5) = e^{-5/1} \approx 0.007$$

8.4 Simulated Annealing – Python Implementation

```
1 import numpy as np
2
3 def simulated_annealing(f, x0, t_init=10.0, t_schedule='boltzmann',
4                         sigma=1.0, n_iter=1000):
5     """
6     f            : objective function
7     x0           : starting point
8     t_init       : initial temperature
9     t_schedule   : 'boltzmann' | 'cauchy' | 'exponential'
10    sigma        : transition distribution std. dev.
11    """
12    x = np.array(x0, dtype=float)
13    y = f(x)
14    best_x, best_y = x.copy(), y
15    gamma = 0.95 # for exponential schedule
16
17    for k in range(1, n_iter + 1):
18        # Temperature schedule
19        if t_schedule == 'boltzmann':
20            t = t_init / np.log(k + 1)
21        elif t_schedule == 'cauchy':
22            t = t_init / k
23        else: # exponential
24            t = t_init * gamma**k
25
26        # Propose new point (Gaussian transition)
27        x_new = x + np.random.randn(len(x)) * sigma
```

8.4 SA – Effect of Temperature and Variance (Example 8.2)

SA on Ackley's Function: σ vs Temperature (Example 8.2)



Key Observations

- **Spread** of the transition distribution (σ) has the greatest impact on performance on Ackley's function.
- Small σ ($= 1$): can't jump between distant local minima.
- Large σ ($= 25$): explores broadly but converges slowly.
- Optimal σ (≈ 5) balances exploration and exploitation.
- Temperature schedule has secondary effect here.

Takeaway

For functions with widely-separated local minima, use a transition distribution with **large spread**.

8.4 Adaptive Simulated Annealing (Algorithm 8.7)

Adaptive SA Idea (Ingber, 1989)

- Each coordinate i has its own step vector v_i and temperature.
- The step vector is adapted so that the acceptance rate stays near a target.
- **Procedure:**
 - 1 Perform a cycle of n_s random moves, one coordinate at a time.
 - 2 Adjust v_i to target 50% acceptance in each direction.
 - 3 Repeat n_t temperature-reduction cycles.
 - 4 When no adjustment needed for n_e cycles, add point to optimum list.

Step-Size Update

$$\left(v_i \left(1 + c_i \frac{A_i / n_s - 0.6}{0.4} \right) \right) \quad \text{if } A_i > 0.6 n_s$$



interval objective function, appendix B.3. The right plot shows the interval objective function values versus their radii, which is useful for identifying intervals to split with further evaluations.

Figure 7.12. The left plot shows

8.5 Cross-Entropy Method

Idea

- Maintain a **proposal distribution** $P_\theta(\mathbf{x})$ over the design space.
- Iteratively fit P to the **elite samples** – the m_{elite} samples with best objective values.
- Reduces cross-entropy between proposal and the target (distribution concentrated at global optimum).

Algorithm 8.8 (CEM)

- 1 Draw m samples: $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)} \sim P$
- 2 Rank samples by f : best m_{elite} become elite set $\mathcal{E}$
- 3 Update $P \leftarrow \text{fit}(\text{type}(P), \mathcal{E})$
- 4 Repeat until convergence

26 CHAPTER 7. DIRECT METHODS

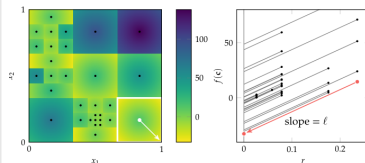


Figure 7.14. The left plot shows the Lipschitz bounds constructed for the RECT intervals using the Lipschitz constant $\ell = 200$, but now ℓ lights the interval containing lowest value. The right plot shows how the lowest lower bound given Lipschitz constant is the with the lowest x -intercept in right-hand plot.

CEM on a 1D function. Distribution narrows onto the global minimum over iterations.

8.5 Cross-Entropy Method – Python Implementation

```
1 import numpy as np
2
3 def cross_entropy_method(f, mu0, Sigma0, k_max=50, m=100, m_elite=10):
4     """
5     f          : objective to MINIMISE  $R^n \rightarrow R$ 
6     mu0        : initial mean (n,)
7     Sigma0     : initial covariance (n,n)
8     m          : total samples per iteration
9     m_elite    : number of elite samples used for refitting
10    Returns    : estimated minimiser (final mean)
11    """
12    mu = np.array(mu0, dtype=float)
13    Sigma = np.array(Sigma0, dtype=float)
14
15    for k in range(k_max):
16        # Sample from proposal distribution
17        samples = np.random.multivariate_normal(mu, Sigma, m) # (m, n)
18        values = np.array([f(x) for x in samples])
19
20        # Select elite samples (lowest f values)
21        elite_idx = np.argsort(values)[:m_elite]
22        elite = samples[elite_idx]
23
24        # Refit distribution (MLE for multivariate Gaussian)
25        mu = elite.mean(axis=0)
26        diff = elite - mu
27        Sigma = (diff.T @ diff) / m_elite + np.eye(len(mu)) * 1e-6
```

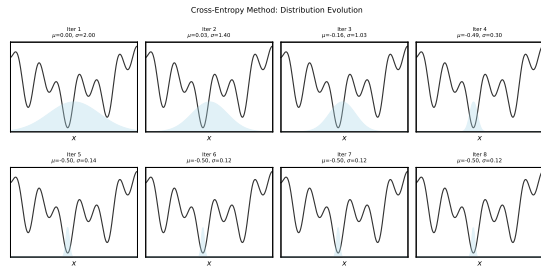
8.5 CEM – Numerical Example (Branin, Example 8.3)

Setup

- Branin function: 3 global minima at $f^* \approx 0.398$
- Initial: $\mu = [0.5, 5.0]$, $\Sigma = \mathbf{I}$
- $m = 40$ samples, $m_{\text{elite}} = 10$, $k_{\text{max}} = 10$

Convergence

Iter	μ_1	μ_2	$f(\mu)$
0	0.50	5.00	51.8
2	-0.12	4.21	12.3
4	-2.1	12.1	3.4
6	-3.1	12.3	0.9
8	-3.14	12.27	0.40
10	-3.14	12.27	0.398



CEM applied to a 1D multi-modal function. The Gaussian proposal (blue shading) narrows onto the global minimum.

8.6 Proposal Distribution Descent (PDD)

Idea

Instead of fitting to elite samples (CEM), **directly differentiate** through the expected objective:

$$J(\theta) = \mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x})]$$

Update: $\theta^{(k+1)} \leftarrow \theta^{(k)} - \alpha \nabla_\theta J(\theta^{(k)})$

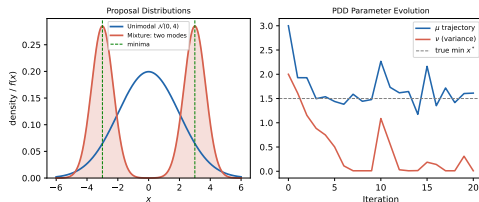
Score Function Gradient (REINFORCE)

Using $\nabla_\theta \log p = \nabla p / p$:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\mathbf{x} \sim p_\theta} [f(\mathbf{x}) \nabla_\theta \log p_\theta(\mathbf{x})]$$

Monte Carlo estimate with m samples:

$$\nabla_\theta J \approx \frac{1}{m} \sum_{i=1}^m f(\mathbf{x}^{(i)}) \nabla_\theta \log p_\theta(\mathbf{x}^{(i)})$$



Left: unimodal vs. mixture proposal. Right: PDD parameter evolution on a noisy objective.

Exercise 8.3 warning

When $\mu = x^*$ (at the optimum), the gradient w.r.t. variance ν is *positive*. A fixed step size α can push ν negative. Use $\nu = e^\eta$ reparameterisation or enforce $\nu > 0$.

8.6 PDD – Python Implementation

```
1 import numpy as np
2
3 def proposal_distribution_descent(f, mu0, nu0, alpha=0.1, m=50, n_iter=200):
4     """
5     Minimise  $E_{x \sim N(\mu, \nu)}[f(x)]$  via score-function gradient.
6     f      : objective  $R \rightarrow R$ 
7     mu0    : initial mean
8     nu0    : initial variance ( $>0$ )
9     """
10    mu, log_nu = float(mu0), np.log(float(nu0)) #  $\nu = \exp(\log_{\nu}) > 0$ 
11
12    for _ in range(n_iter):
13        nu = np.exp(log_nu)
14        xs = np.random.normal(mu, np.sqrt(nu), m)
15        fs = np.array([f(x) for x in xs])
16
17        # Score-function gradients (Alg 8.9 eq 8.16)
18        score_mu = (xs - mu) / nu
19        score_nu = ((xs - mu)**2 - nu) / (2 * nu**2)
20
21        g_mu = np.mean(fs * score_mu)
22        g_nu = np.mean(fs * score_nu)
23
24        # Update (reparameterise  $\nu = \exp(\log_{\nu})$  to keep  $\nu > 0$ )
25        g_log_nu = g_nu * nu # chain rule
26        mu -= alpha * g_mu
27        log_nu -= alpha * g_log_nu
```

8.7 Information-Geometric Optimization

Motivation: Invariance

- Ordinary gradient descent on θ is **not invariant** to reparameterisation.
- **Natural gradient** uses the *Fisher information matrix* $\mathbf{F}$ to correct for geometry of the statistical manifold:

$$\tilde{\nabla}_{\theta} J = \mathbf{F}^{-1}(\theta) \nabla_{\theta} J(\theta)$$

$$\mathbf{F}(\theta) = \mathbb{E}_{\mathbf{x} \sim p_{\theta}} [\nabla_{\theta} \log p(\nabla_{\theta} \log p)^{\top}]$$

Natural Gradient for $\mathcal{N}(\mu, \Sigma)$

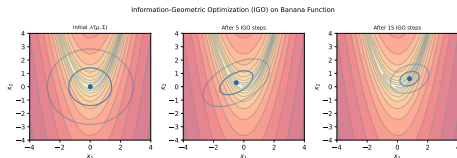
Log-density:

$$\log p(\mathbf{x}|\mu, \Sigma) = -\frac{1}{2}(n \log 2\pi + \log |\Sigma| + (\mathbf{x} - \mu)^{\top} \Sigma^{-1} (\mathbf{x} - \mu))$$

Natural gradient updates (eqs. 8.25–8.26):

$$\tilde{\nabla}_{\mu} J = \mathbb{E}[f(\mathbf{x})(\mathbf{x} - \mu)]$$

$$\tilde{\nabla}_{\Sigma} J = -\frac{1}{2} \mathbb{E}[f(\mathbf{x})(\mathbf{x} - \mu)(\mathbf{x} - \mu)^{\top} - \Sigma)]$$



IGO distributions converging on the Banana (Rosenbrock) function. Fisher information removes scale dependence.

Why IGO is Better

- Invariant to **order-preserving transforms** of f .
- No tuning of absolute scale needed.
- Uses **fitness ranking** rather than raw values.

8.7 IGO – Python Implementation (Algorithm 8.10)

```
1 import numpy as np
2
3 def information_geometric_optimization(f, mu0, Sigma0, k_max=100, alpha=1.0):
4     """
5     Algorithm 8.10: IGO with multivariate normal.
6     Sample size  $m = 4 + \text{floor}(3 \cdot \log(n))$ .
7     Elite fraction: half the samples (ranked weighting).
8     """
9     mu = np.array(mu0, dtype=float)
10    Sigma = np.array(Sigma0, dtype=float)
11    n = len(mu)
12
13    m = 4 + int(3 * np.log(n))          # population size
14    m_elite = m // 2                    # number of elite samples
15
16    # Log-based weights (eq. 8.29): rank 1 = best
17    ws = np.log(m_elite + 1) - np.log(np.arange(1, m_elite + 1))
18    ws /= ws.sum()                     # normalise
19
20    for _ in range(k_max):
21        # Sample and evaluate
22        xs = np.random.multivariate_normal(mu, Sigma, m)
23        ys = np.array([f(x) for x in xs])
24        order = np.argsort(ys)         # best to worst
25
26        # Weighted natural gradient updates
27        delta_mu = np.zeros(n)
```

8.8 Covariance Matrix Adaptation Evolution Strategy

Extension of IGO

CMA-ES (Hansen & Ostermeier, 2001) extends IGO by separating:

- σ – overall step size (scale)
- $\mathbf{C}$ – shape matrix (covariance up to scale)

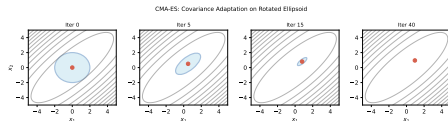
$$\mathbf{x}^{(i)} \sim \mathcal{N}(\boldsymbol{\mu}, \sigma^2 \mathbf{C})$$

Mean Update (same as IGO)

$$\boldsymbol{\mu}' = \boldsymbol{\mu} + \alpha_{\mu} \sum_{i=1}^m w_i (\mathbf{x}^{[i]} - \boldsymbol{\mu})$$

where $[i]$ indexes samples ordered best-to-worst and w_i are log weights.

Evolution Paths



CMA-ES adapts both scale (σ) and shape ($\mathbf{C}$) to follow the problem geometry.

Effective Population Size

$$m_{\text{eff}} = \frac{1}{\sum_{i=1}^m w_i^2}$$

Controls the variance of the weighted mean.

8.8 CMA-ES – Step Size and Covariance Updates

Step-Size Control (eq. 8.34)

$$\sigma^{(k+1)} = \sigma^{(k)} \exp\left(\frac{\alpha_\sigma}{\beta_\sigma} \left(\frac{\|\mathbf{p}_\sigma\|}{\mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|]} - 1\right)\right)$$

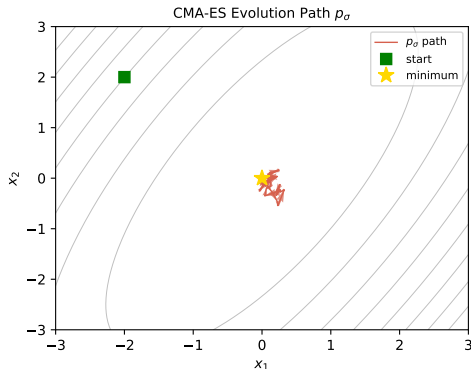
- $\mathbb{E}[\|\mathcal{N}(0, \mathbf{I})\|] = \sqrt{2} \Gamma(\frac{n+1}{2}) / \Gamma(\frac{n}{2})$
- If $\|\mathbf{p}_\sigma\| > \text{expected} \Rightarrow \text{increase } \sigma$
- If $\|\mathbf{p}_\sigma\| < \text{expected} \Rightarrow \text{decrease } \sigma$

Covariance Update (eq. 8.35)

Rank-1 + rank- m update:

$$\mathbf{C}^{(k+1)} = (1 - \alpha_1 - \alpha_e) \mathbf{C}^{(k)} + \alpha_1 \mathbf{p}_c \mathbf{p}_c^\top + \frac{\alpha_e}{\sigma^2} \sum_{i=1}^m w_i \mathbf{x}^{[i]} (\mathbf{x}^{[i]})^\top$$

- Rank 1: exploits correlated steps over time



Evolution path $\mathbf{p}_\sigma$ adapts step direction over iterations.

Default Hyperparameters (n -dim)

$$m = 4 + \lfloor 3 \ln n \rfloor$$

$$m_e = m/2$$

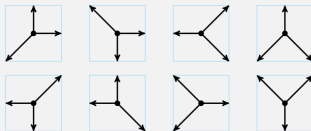
8.8 CMA-ES – Python Implementation (Algorithm 8.11)

```
1 import numpy as np
2 from scipy.special import gamma as gamma_fn
3
4 def cmaes(f, mu0, sigma0=1.0, C0=None, k_max=500):
5     n = len(mu0)
6     mu = np.array(mu0, dtype=float)
7     sig = float(sigma0)
8     C = np.eye(n) if C0 is None else np.array(C0, dtype=float)
9
10    m = 4 + int(3 * np.log(n))
11    m_elite = m // 2
12    a1 = 2 / n**2;          ae = m_elite / n**2
13    ac = 1 / np.sqrt(n);    ao = 1 / np.sqrt(n)
14    amu = 1 - a1 - ae;      bs = 1.0
15    ws = np.log(m_elite+1) - np.log(np.arange(1,m_elite+1))
16    ws /= ws.sum()
17    m_eff = 1 / np.sum(ws**2)
18    exp_len = (np.sqrt(2) * gamma_fn((n+1)/2) / gamma_fn(n/2))
19    p_sig = np.zeros(n);    p_c = np.zeros(n)
20
21    for _ in range(k_max):
22        xs = np.random.multivariate_normal(mu, sig**2 * C, m)
23        ys = np.array([f(x) for x in xs])
24        idx = np.argsort(ys)          # best first
25
26        mu_new = mu + amu * sum(ws[i]*(xs[idx[i]]-mu) for i in range(m_elite))
27        delta = (mu_new - mu) / sig
```

8.8 CMA-ES – Flower Function Example

8.2. MESH ADAPTIVE DIRECT SEARCH 137

Consider positive spanning sets constructed from the nonzero directions $d_1, d_2 \in \{-1, 0, 1\}$. There are 8 positive spanning sets with 3 elements that can be constructed from these directions:



Example 8.1. Positive spanning sets for $\mathbb{R}^2$. Note that the lower triangular generation strategy can only generate the first two columns of spanning sets.

The rows and columns of L are then randomly permuted to obtain a matrix D whose columns correspond to n directions that linearly span $\mathbb{R}^n$. Each of these directions is inscribed inside a hypercube of side length $2/\sqrt{\alpha^{(k)}}$.⁷

Two common methods for obtaining a positive spanning set from the linearly spanning set are to add one additional direction $d^{(n+1)} = -\sum_{i=1}^n d^{(i)}$ and to add n additional directions $d^{(n+j)} = -d^{(j)}$ for j in $1 : n$. We use the first method in algorithm 8.2.

The step size α starts at 1, is always a power of 4, and never exceeds 1. Using a power of 4 causes the maximum possible componentwise step size taken in each iteration to be scaled by a factor of 2, as the maximum componentwise step size $\alpha/\sqrt{\alpha}$ has length $4^m/\sqrt{4^m} = 2^m$ for integer $m < 1$. The step size is updated according to:

$$\alpha^{(k+1)} \leftarrow \begin{cases} \alpha^{(k)}/4 & \text{if no improvement was found in this iteration} \\ \min(1, 4\alpha^{(k)}) & \text{otherwise} \end{cases} \quad (8.4)$$

Mesh adaptive direct search is opportunistic and does not support dynamic ordering⁸ because, after a successful iteration, the new random pattern may not contain the previous successful direction. The algorithm queries a new design point along the accepted descent direction. If $f(\mathbf{x}^{(k)} + \alpha \mathbf{d}) < f(\mathbf{x}^{(k-1)})$,

⁷ For each direction, $\|\mathbf{d}^{(i)}\|_\infty = 1/\sqrt{\alpha^{(k)}}$.

⁸ See section 7.4.

Observations

- Iteration 0: large isotropic distribution.
- Iteration 5: ellipse aligns with principal curvature direction.
- Iteration 15: covariance shrinks; distribution concentrates near minimum.
- Iteration 40: near-converged, tightly around global minimum.

Scale Invariance

CMA-ES is invariant to:

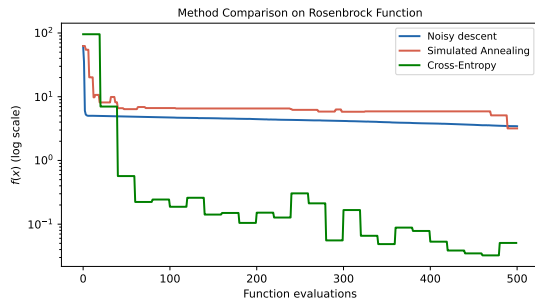
- 1 **Order-preserving transformations** of f (uses ranks not values).
- 2 **Rotation** of the search space.
- 3 **Scaling** of the search space (via σ adaptation).

Comparing Stochastic Methods

Summary Table

Method	Order	Memory	Global?
Noisy Descent	1st	$O(n)$	No
MADS	0th	$O(n^2)$	✓ [†]
MeZO	0th	$O(1)$	No
Sim. Annealing	0th	$O(1)$	✓ [†]
Cross-Entropy	0th	$O(n^2)$	✓
PDD	0th	$O(n^2)$	No
IGO	0th	$O(n^2)$	✓
CMA-ES	0th	$O(n^2)$	✓

† asymptotic / with conditions



Noisy descent, SA, and CEM on the Rosenbrock function from $(-1.5, 1.5)$.

When to Use What

- **Gradient available, large scale:** Noisy descent, MeZO
- **Black box, $n < 20$:** MADS, SA
- **Many local minima:** SA, CEM, CMA-ES

8.9 Summary I

- **Stochastic methods** employ random numbers during optimisation to escape local minima and handle noisy objectives.
- **Noisy descent** adds Gaussian noise to the gradient. Convergence is guaranteed by the Robbins–Monro step-size conditions.
- **Mesh adaptive direct search (MADS)** is a pattern-search method that uses randomly generated positive spanning sets and an adaptive step size that is always a power of 4.
- **Memory-efficient zeroth-order optimisation (MeZO)** estimates the gradient using only two forward passes and a re-seeded PRNG, requiring $O(1)$ additional memory.
- **Simulated annealing** uses a temperature-controlled Metropolis criterion to accept uphill moves with probability $e^{-\Delta y/t}$. The Boltzmann schedule ($t \propto 1/\ln k$) guarantees asymptotic convergence; faster schedules are used in practice.
- **The cross-entropy method** maintains a proposal distribution and repeatedly fits it to the elite samples (best function values) using maximum likelihood estimation.
- **Proposal distribution descent** uses score-function (REINFORCE) gradient estimates to descend on the expected objective directly.

8.9 Summary II

- **Information-geometric optimisation (IGO)** applies the natural gradient, pre-multiplied by the inverse Fisher information matrix, making updates invariant to the parameterisation of the distribution and to order-preserving transformations of f .
- **Covariance matrix adaptation (CMA-ES)** extends IGO by separately tracking a global step size σ and a shape matrix $\mathbf{C}$. Evolution paths accumulate direction information across iterations, enabling efficient learning of the covariance structure.

8.10 Exercises I

Exercise 8.1

We have shown that mixture proposal distributions can better capture multiple minima. Why might their use in the cross-entropy method be limited?

Solution: The cross-entropy method must fit distribution parameters with every iteration. Unfortunately, no known analytic solutions for fitting multivariate mixture distributions exist. One commonly uses the iterative expectation-maximisation (EM) algorithm, which itself requires many iterations to converge.

Exercise 8.2

In the cross-entropy method, what is a potential effect of using an elite sample size that is very close to the total sample size?

Solution: If $m_{\text{elite}}/m \rightarrow 1$, the resulting distribution will closely match the population. There will not be a significant bias towards the best locations for a minimiser, and so convergence will be slow.

8.10 Exercises II

Exercise 8.3

Consider proposal distribution descent with $\mathcal{N}(\mu, \nu)$, with the mean on the optimum $\mu = x^*$. Show that the update for variance ν becomes increasingly negative as $\nu \rightarrow 0$, and explain why a fixed step factor α poses a numerical risk.

Solution: The gradient of J w.r.t. ν at the optimum approximates $f(x) \approx \frac{1}{2}h(x - \mu)^2$, $h = f''(\mu) > 0$. Substituting:

$$\nabla_{\nu}\mathbb{E}[f] \approx \frac{h}{2} \left(-\frac{1}{2\nu}\mathbb{E}[z^2] + \frac{1}{2\nu^2}\mathbb{E}[z^4] \right) = \frac{h}{2} \left(-\frac{1}{2\nu} + \frac{3}{2\nu} \right) = \frac{h}{2} > 0$$

The gradient is positive (descent reduces ν). As $\nu \rightarrow 0$, the term $\partial_{\nu} \log \mathcal{N} \propto (x - \mu)^2 / (2\nu^2)$ grows as $O(1/\nu)$, so the gradient estimate has large variance. For any fixed α , eventually $\nu - \alpha \nabla_{\nu} J < 0$. Fix: use $\nu = e^{\eta}$ (reparameterisation).

8.10 Exercises III

Exercise 8.4

Derive the MLE for the cross-entropy method using multivariate normal distributions, obtaining:

$$\boldsymbol{\mu}^{(k+1)} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}^{(i)}, \quad \boldsymbol{\Sigma}^{(k+1)} = \frac{1}{m} \sum_{i=1}^m (\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)})(\mathbf{x}^{(i)} - \boldsymbol{\mu}^{(k+1)})^\top$$

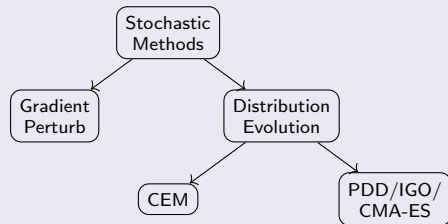
Solution sketch: Maximise the log-likelihood $\ell(\boldsymbol{\mu}|\mathbf{x}^{(1:m)}) = \sum_i \log p(\mathbf{x}^{(i)}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Taking $\nabla_{\boldsymbol{\mu}} \ell = 0$ gives $\boldsymbol{\mu} = \frac{1}{m} \sum_i \mathbf{x}^{(i)}$. Similarly, letting $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ and maximising ℓ w.r.t. $\boldsymbol{\Lambda}$ gives $\boldsymbol{\Sigma} = \frac{1}{m} \sum_i (\mathbf{x}^{(i)} - \boldsymbol{\mu})(\mathbf{x}^{(i)} - \boldsymbol{\mu})^\top$.

Conclusion & Key Takeaways

Chapter 8 in One Slide

- 1 **Noise is a tool** – not just a nuisance. Random perturbations help escape local optima.
- 2 **Temperature as exploration parameter** – SA shows how to smoothly transition from exploration to exploitation.
- 3 **Distribution-based methods** (CEM, PDD, IGO, CMA-ES) turn optimisation into inference: find the distribution concentrated at the minimum.
- 4 **Natural gradient** (IGO, CMA-ES) provides invariance properties essential for robust black-box optimisation.
- 5 **MeZO** bridges stochastic methods and modern large-scale ML where memory is the bottleneck.

Hierarchy of Methods



Next Chapter

Chapter 9: Population Methods (Genetic Algorithms, Particle Swarm, etc.) – extend these ideas to maintain a *diverse population* of solutions.